

Problem Statement No. 13

AI-Powered Personalized Study Assistant

1. Background

Students today face the challenge of processing large volumes of study material efficiently. Traditional study methods are time-consuming and often fail to adapt to individual learning needs. With advances in Generative AI and Large Language Models (LLMs), it is now possible to build intelligent assistants that can read, understand, and transform academic content into personalised study tools automatically.

2. Problem Statement

Design and develop an AI-powered personalised study assistant that enables college students to upload their study material (PDFs, notes, textbook images) and receive automated, intelligent academic support powered by IBM Granite Large Language Models via IBM watsonx.ai.

3. Objectives

- Automatically generate concise chapter summaries from uploaded documents.
- Create interactive MCQ quizzes with answer keys and explanations.
- Generate term-definition flashcard pairs for active recall practice.
- Build a personalised study planner based on exam date and available hours.
- Enable RAG-powered Q&A; — answer student questions from uploaded material.
- Track progress, identify weak areas, and display a visual dashboard.
- Integrate IBM Granite models via watsonx.ai for all AI generation tasks.

4. Scope of Work

The solution must be a fully functional web application with the following components:

- Document Upload Module — PDF, plain text, Markdown, and image (OCR) support.
- Summary Agent — IBM Granite-powered chapter summarisation.
- Quiz Agent — MCQ generation with A/B/C/D options and answer keys.
- Flashcard Agent — Term-definition pair extraction.
- Study Planner Agent — Day-by-day schedule generation with weak area weighting.
- RAG Q&A; Module — ChromaDB vector store + IBM Slate embeddings + Granite answering.
- Progress Dashboard — Plotly charts for activity tracking and milestone display.

- IBM Langflow Integration — Visual orchestration of all agent pipelines.

5. Required Technology Stack

Component	Technology
AI Models	IBM Granite (via watsonx.ai)
Agent Orchestration	IBM Langflow
Embeddings (RAG)	IBM Slate-125M (watsonx.ai)
Vector Database	ChromaDB (local persistent)
Frontend	Streamlit (Python)
PDF Parsing	PyMuPDF (fitz)
Image OCR	Tesseract / pytesseract
Backend Language	Python 3.11+

6. Expected Deliverables

- Fully functional Streamlit web application.
- All four AI agents implemented and tested.
- IBM Langflow pipeline exported as JSON.
- Source code uploaded to a public GitHub repository.
- Project report and presentation (PPT).
- Demo video / screenshots of working application.

7. Evaluation Criteria

Criterion	Weightage
Functionality & Completeness	30%
IBM Granite / watsonx.ai Integration	25%
Code Quality & Documentation	20%
UI/UX & User Experience	15%
Presentation & Viva	10%